

Page No.
 Date.
 1] Title :- Hospital patient management System
Using Doubly Linked List

2] Problem Statement :- Hospitals need to maintain a large number of patient records containing details such as patient ID, Name, Age, and Disease. Managing these records manually or using fixed sized arrays can make insertion, searching, and deletion difficult. Therefore, a Hospital patient management System is developed using a doubly linked list. The System allows hospital staff to add new patient using patient ID, and delete the record when the patient is discharged.

3] Objectives

- i) to develop a simple system for managing hospital patient records.
- ii) to implement a doubly linked list using C++
- iii) to store patient ID, Name, Age, and Disease dynamically.
- iv) to provide an operating operation for adding new patients.
- v) to display all available patient records
- vi) to search for a patient using patient ID
- vii) to delete a patient's record after discharge

Page No.
 Date:
 Niii) To understand the practical application of prev and next pointers.

4] Algorithms

Algorithm 1: Add a new patient

Step 1: Start

Step 2: Create a newNode using dynamic memory allocation

Step 3: Enter patient ID, Name, Age, and Disease

Step 4: Set newNode $\rightarrow$ prev = NULL and newNode $\rightarrow$ next = NULL

Step 5: Check whether head = NULL.

- if yes, make head = newNode.
- otherwise, traverse the list to the last node

Step 6: Connect the new node with the last node:

- last $\rightarrow$ next = newNode
- newNode $\rightarrow$ prev = last

Step 7: Display "patient added successfully"

Step 8: Stop

Algorithm 2: Display Patient records

Step 1: Start

Step 2: Check whether $head == NULL$

- if yes, display "No patient record found".
- otherwise, continue.

Step 3: Set $temp = head$.

Step 4: Display the patient information.

Step 5: Move to the next node using $temp = temp \rightarrow next$.

Step 6: Repeat until $temp == NULL$

Step 7: Stop

Algorithm 3: Search Patient

Step 1: Start

Step 2: Check whether $head == NULL$.

- if

Step 2: Enter the patient ID to be searched.

Step 3: Set $temp = head$.

Step 4: Compare $temp \rightarrow$ Patient ID with entered ID

- if equal, display the patient details and stop searching.

- otherwise, move to the next node

Algorithm 4: Delete patient After Discharge.

Step 1: Start.

Step 2: Enter the patient ID to be deleted

Step 3: Search for the node containing that patient ID.

Step 4: if the patient is not found, display patient not found.

Step 5: if the node is the first node update head.

Step 6: Otherwise, connect the previous node to the next node.

$\text{temp} \rightarrow \text{prev} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$

Step 7: if a next node exists, update its previous pointer.

$\text{temp} \rightarrow \text{next} \rightarrow \text{prev} = \text{temp} \rightarrow \text{prev}$

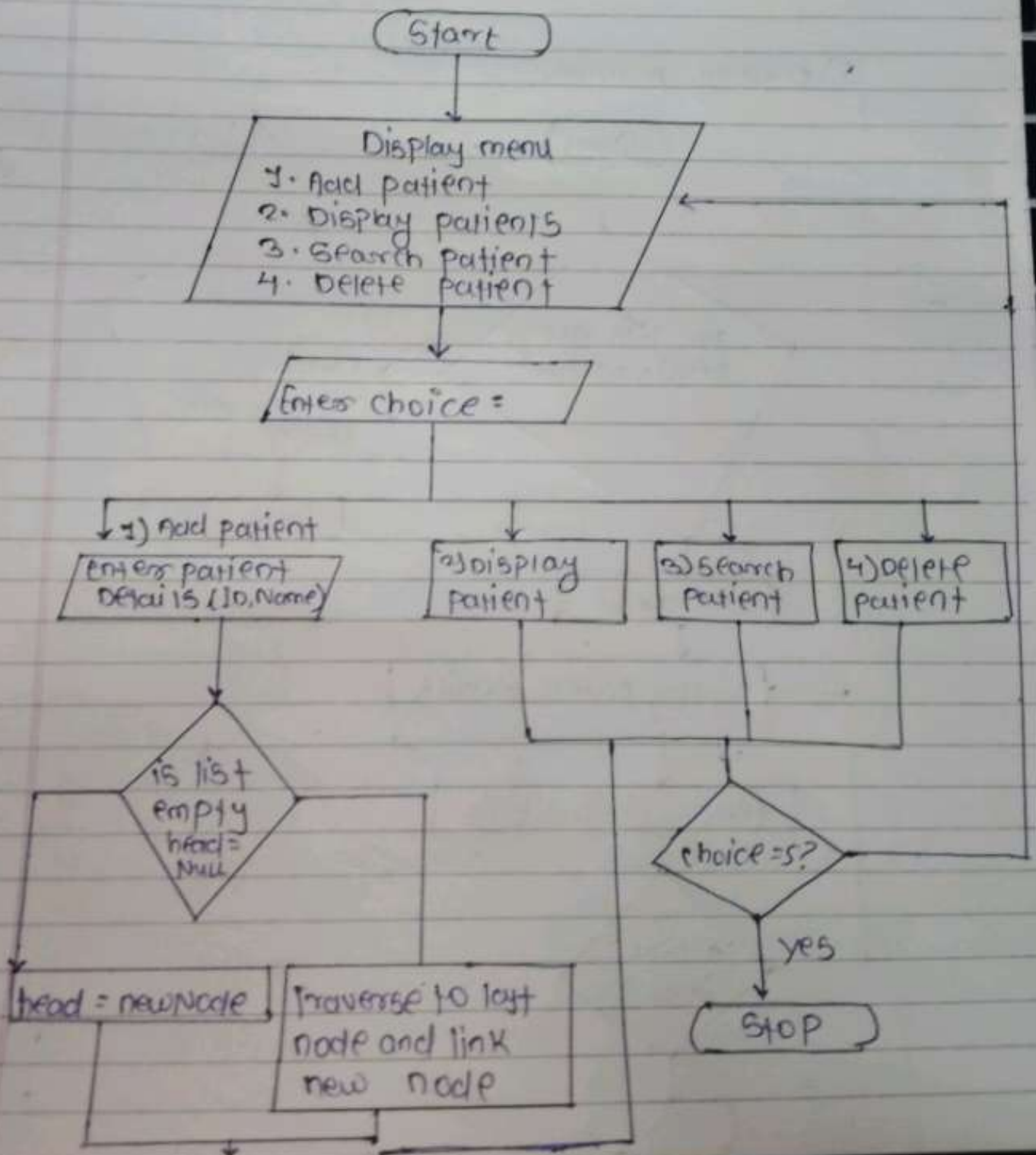
Step 8: Delete the node using delete temp.

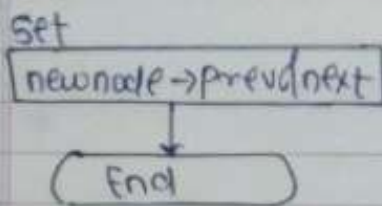
Step 9: Display "patient discharged successfully"

Step 10: Stop

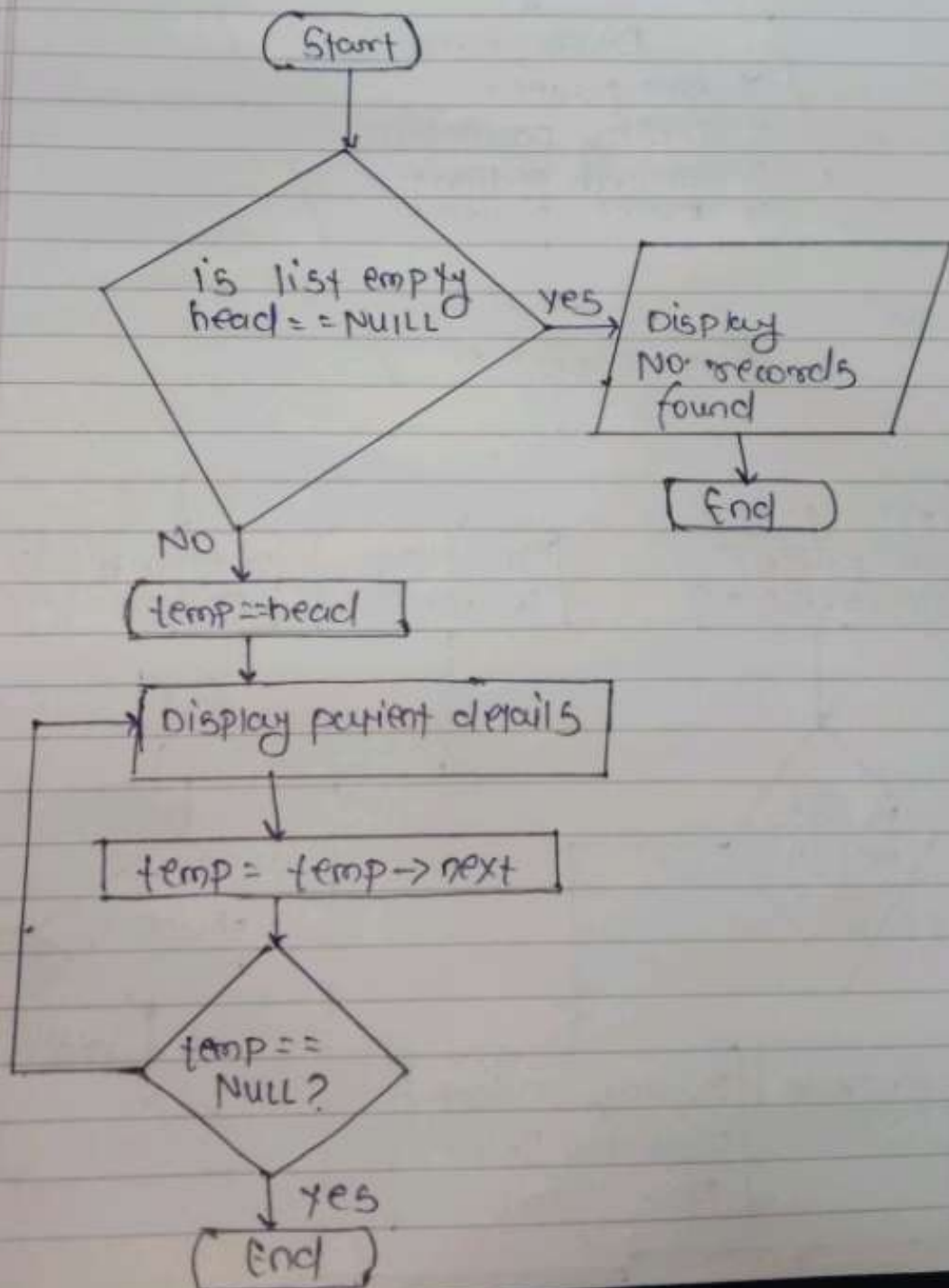
5] flowchart

1. main program flowchart

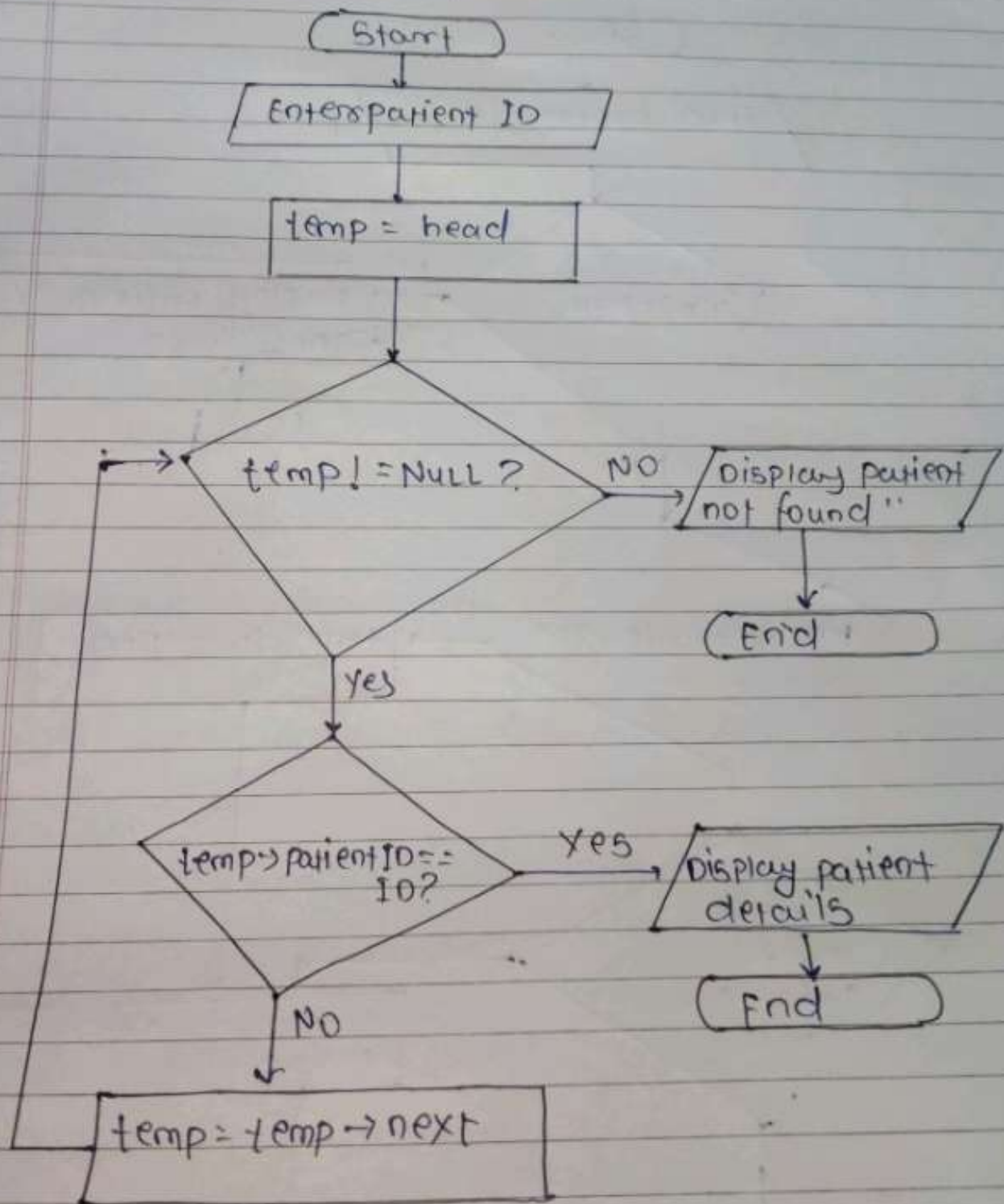




2) display patients



3) Search patient flowchart.



4) delete patient flowchart

